AI & HEALTH CARE

칼로리 소모량 예측 모델

머리어깨 8조

김서영

돈유정

진태준

목차

1. 데이터 로드 및 탐색 (EDA)

2. 데이터 전처리

3. 모델링

4. 모델 해석 및 시각화

5. 고찰

6. 인용 및 소감

1. 데이터 로드 및 탐색 (EDA)

1.1 필수 라이브러리 호출 및 데이터 로드

```python
# 설치
!pip install optuna catboost shap koreanize-matplotlib -q # optuna: 하이퍼파라미터 최적화 라이브러리
                                                          # catboost: Gradient Boosting 기반 머신러닝 모델 (카테고리 데이터 처리 강점)
                                                          # shap: 모델 해석용 라이브러리 (변수 중요도 시각화)
                                                          # koreanize-matplotlib: matplotlib에서 한글 폰트 깨짐 방지
                                                          # -q: 설치 메시지를 최소화

# 라이브러리
import numpy as np # 수치 계산, 배열 연산
import pandas as pd # 데이터 처리, DataFrame
from scipy import stats # 통계 함수 (t-test, z-score 등)
from sklearn.ensemble import GradientBoostingRegressor # GBM 모델
from sklearn.model_selection import KFold # 교차검증
from sklearn.metrics import mean_squared_error # 평가 지표 (RMSE 계산)
from numpy.linalg import lstsq  # 최소제곱 해 계산 (선형 회귀)
from catboost import CatBoostRegressor # CatBoost 회귀 모델
import optuna, warnings, shap # 하이퍼파라미터 최적화, 경고 무시, SHAP 값 계산 (모델 설명)
import koreanize_matplotlib  # 한글 폰트 자동 설정
import matplotlib.pyplot as plt# 그래프 시각화

warnings.filterwarnings('ignore') # 불필요한 경고 무시
optuna.logging.set_verbosity(optuna.logging.WARNING) # Optuna 경고 최소화
```

```
...     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 413.9/413.9 kB 7.9 MB/s eta 0:00:00
        ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 97.1/97.1 MB 8.1 MB/s eta 0:00:00
        ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.9/7.9 MB 46.8 MB/s eta 0:00:00
```

1.2 데이터 로드 & 기본 피처 탐색

```python
# Google Colab 환경에서 로컬 파일 업로드
from google.colab import files
uploaded = files.upload() # 로컬에서 파일 선택 후 Colab으로 파일 업로드

# pandas 라이브러리로 CSV 파일 읽기
import pandas as pd
train = pd.read_csv('train.csv') # 학습 데이터
test  = pd.read_csv('test.csv')  # 테스트 데이터

print("파일 불러오기 완료!")

# Height, Weight, Body Temperature 단위 통일
for df in [train, test]:
    # 키 단위 변환: Feet + Inches → cm
    df['Height_cm']          = df['Height(Feet)']*30.48 + df['Height(Remainder_Inches)']*2.54
    # 몸무게 단위 변환: lb → kg
    df['Weight_kg']          = df['Weight(lb)'] * 0.453592
    # 체온 단위 변환: ℉ → ℃
    df['Body_Temperature_C'] = (df['Body_Temperature(F)'] - 32) * 5/9
    # 불필요한 원본 컬럼 제거
    df.drop(columns=['Height(Feet)','Height(Remainder_Inches)',
                     'Weight(lb)','Body_Temperature(F)'], inplace=True)

# Feature/Target 분리
y      = train['Calories_Burned'].values # 예측할 값 → 타겟
X      = train.drop(columns=['Calories_Burned','ID'])
X_test = test.drop(columns=['ID'])

assert X.isnull().sum().sum() == 0
assert (X['Exercise_Duration'] > 0).all()
print("✅ 데이터 검증 통과")
```

```
파일 선택 파일 2개
test.csv(text/csv) - 434935 bytes, last modified: 2026. 2. 11. - 100% done
train.csv(text/csv) - 482579 bytes, last modified: 2026. 2. 11. - 100% done
Saving test.csv to test (5).csv
Saving train.csv to train (5).csv
파일 불러오기 완료!
✅ 데이터 검증 통과
```

```python
# 데이터 기본 정보 : 각 컬럼명, 데이터 타입, null 여부, 총 데이터 수 확인
print("\n--- 데이터 정보 ---")
train.info()

# 결측치 확인
print("\n--- 결측치 확인 ---")
print(train.isnull().sum())
```

```
--- 데이터 정보 ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7500 entries, 0 to 7499
Data columns (total 11 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   ID                  7500 non-null   object
 1   Exercise_Duration   7500 non-null   float64
 2   BPM                 7500 non-null   float64
 3   Weight_Status       7500 non-null   object
 4   Gender              7500 non-null   object
 5   Age                 7500 non-null   int64
 6   Calories_Burned     7500 non-null   float64
 7   Height_cm           7500 non-null   float64
 8   Weight_kg           7500 non-null   float64
 9   Body_Temperature_C  7500 non-null   float64
 10  AgeGroup            7500 non-null   category
dtypes: category(1), float64(6), int64(1), object(3)
memory usage: 593.6+ KB

--- 결측치 확인 ---
ID                    0
Exercise_Duration     0
BPM                   0
Weight_Status         0
Gender                0
Age                   0
Calories_Burned       0
Height_cm             0
Weight_kg             0
Body_Temperature_C    0
AgeGroup              0
dtype: int64
```

1.2 데이터 로드 & 기본 피처 탐색

```python
# 수치형 변수 통계값 + 이상치 확인

# 수치형 칼럼만 선택
num_cols = train.select_dtypes(include=['int64','float64']).columns.tolist()
target = 'Calories_Burned'
if target in num_cols:
    num_cols.remove(target) # 타켓 제외

# 기본 통계량 계산 (mean, std, min, 25%, 50%, 75%, max)
desc = train[num_cols].describe().T

# IQR 기반 이상치 계산
Q1 = train[num_cols].quantile(0.25)
Q3 = train[num_cols].quantile(0.75)
IQR = Q3 - Q1
outlier_count = ((train[num_cols] < (Q1 - 1.5*IQR)) | (train[num_cols] > (Q3 + 1.5*IQR))).sum()

# 결측치와 이상치 컬럼 추가
desc['missing'] = train[num_cols].isnull().sum()
desc['outliers'] = outlier_count

# 보기 쉽게 컬럼 순서 정리정렬
desc = desc[['missing','outliers','mean','std','min','25%','50%','75%','max']]
print("\n===== 수치형 변수 통계 + 결측치 + 이상치 =====")
print(desc.round(2))  # 소수점 2자리
```

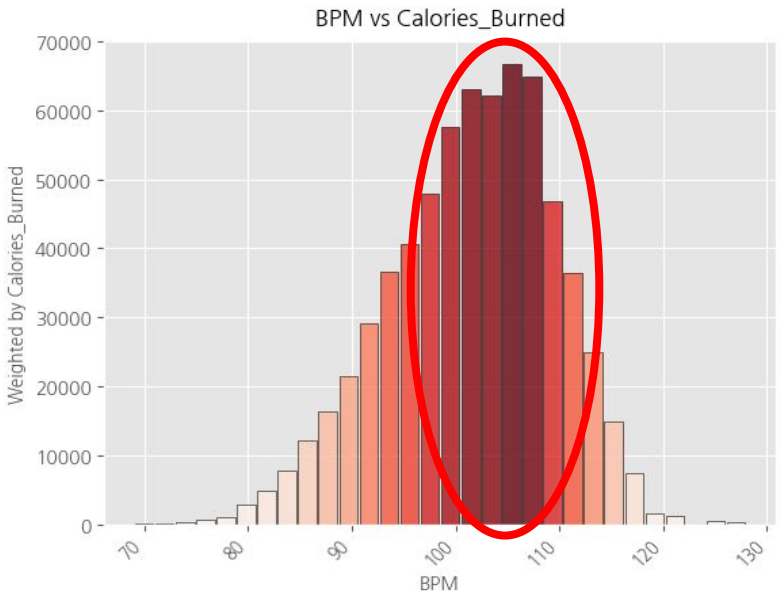
```
...
===== 수치형 변수 통계 + 결측치 + 이상치 =====
                     missing  outliers    mean    std     min     25%     50%  \
Exercise_Duration          0         0   15.50   8.36    1.00    8.00   15.00
BPM                        0         1   95.50   9.59   69.00   88.00   95.00
Age                        0         0   42.64  16.88   20.00   28.00   39.00
Height_cm                  0        12  174.51  14.32  121.92  165.10  175.26
Weight_kg                  0         4   75.01  15.11   36.02   63.00   73.98
Body_Temperature_C         0       195   40.02   0.78   37.11   39.61   40.22

                       75%     max
Exercise_Duration    23.00   30.00
BPM                 103.00  128.00
Age                  56.00   79.00
Height_cm           185.42  218.44
Weight_kg            87.00  132.00
Body_Temperature_C   40.61   41.50
```

```python
# 범주형 변수 통계값 (상위 5개)
cat_cols = train.select_dtypes(include=['object','category']).columns.tolist()
if 'ID' in cat_cols:
    cat_cols.remove('ID') # ID 컬럼 제외

print("\n===== 범주형 변수 분포 (상위 5개) =====")
for col in cat_cols:
    print(f"\n[{col}]")
    print(train[col].value_counts().head(5))
```

```
...
===== 범주형 변수 분포 (상위 5개) =====

[Weight_Status]
Weight_Status
Normal Weight    4681
Overweight       2614
Obese             205
Name: count, dtype: int64

[Gender]
Gender
F    3769
M    3731
Name: count, dtype: int64

[AgeGroup]
AgeGroup
21-30    2105
31-40    1569
60+      1447
41-50    1180
51-60     945
Name: count, dtype: int64
```

1.3 히스토그램 (수치형)

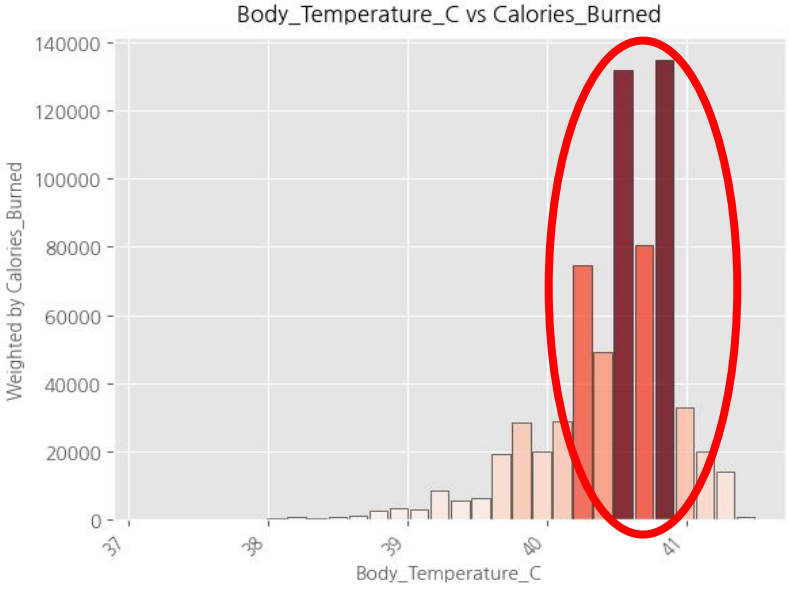
Exercise_Duration vs Calories_Burned
Weighted by Calories_Burned
Exercise_Duration


BPM vs Calories_Burned
Weighted by Calories_Burned
BPM

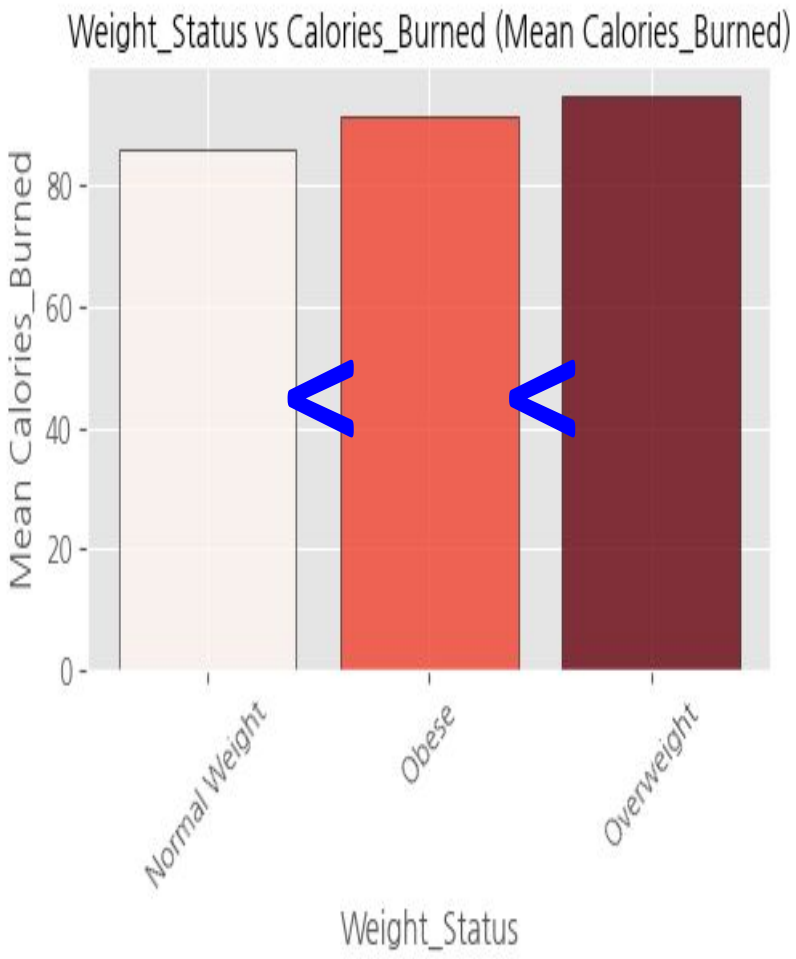
Age vs Calories_Burned
Weighted by Calories_Burned
Age

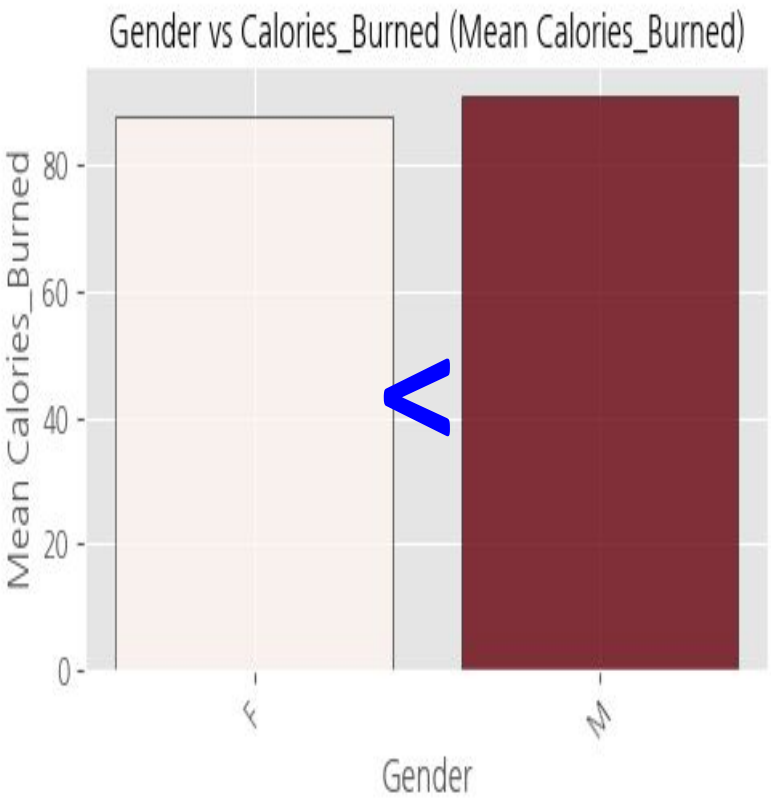
Height_cm vs Calories_Burned
Weighted by Calories_Burned
Height_cm

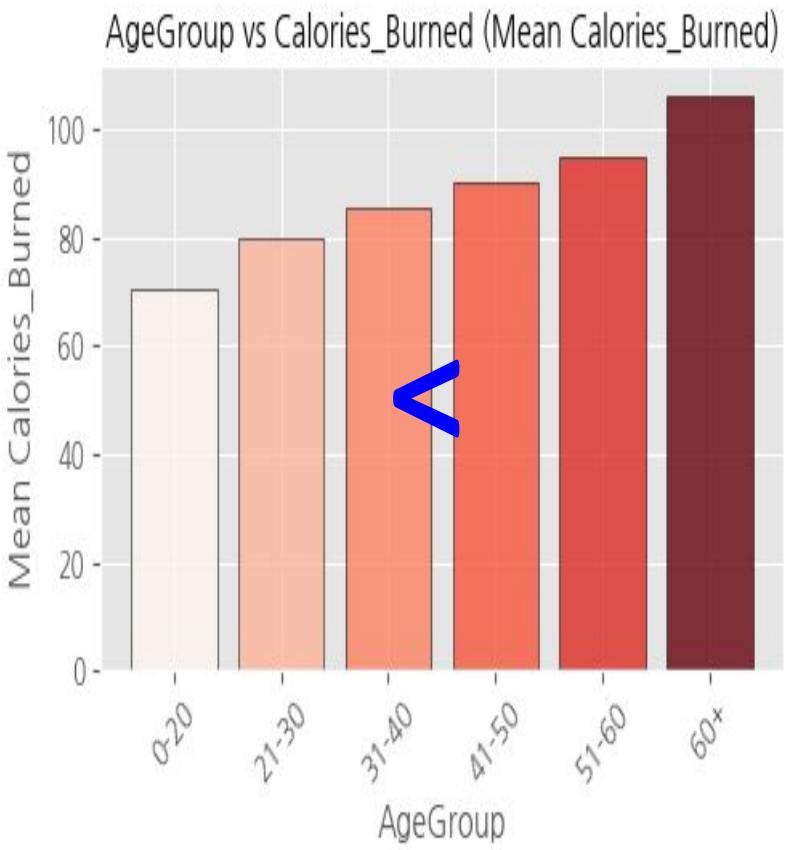
Weight_kg vs Calories_Burned
Weighted by Calories_Burned
Weight_kg


Body_Temperature_C vs Calories_Burned
Weighted by Calories_Burned
Body_Temperature_C

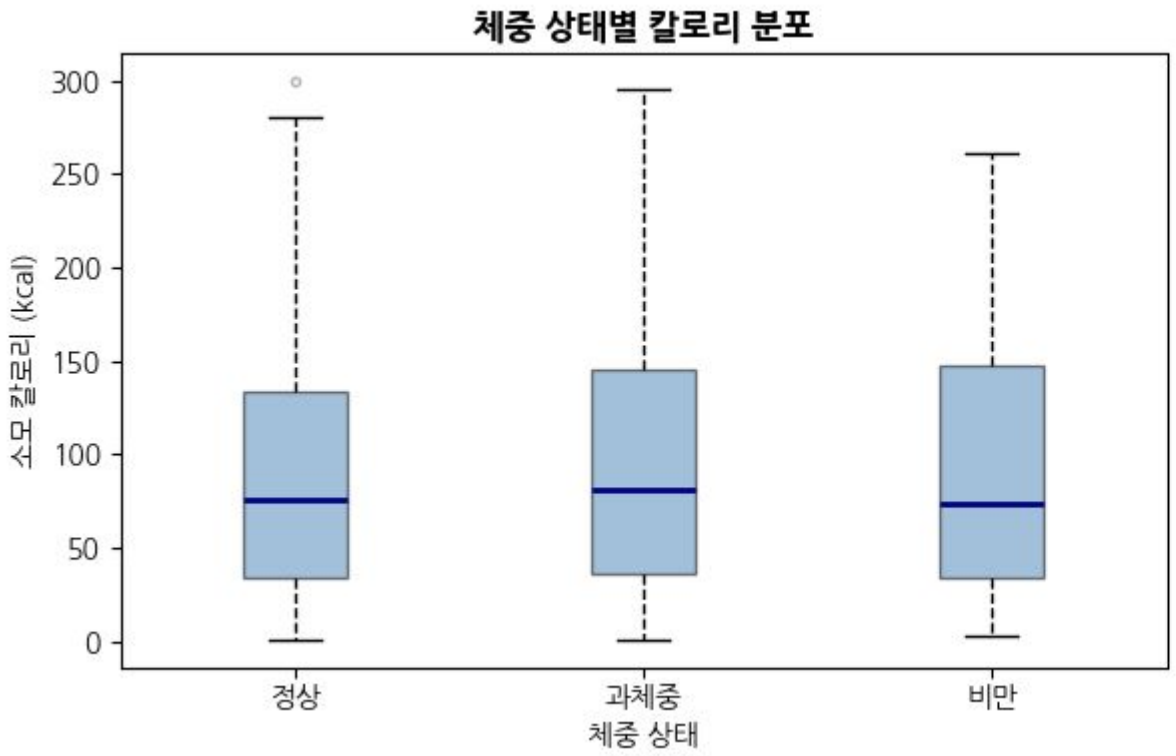
1.3 히스토그램 (범주형)

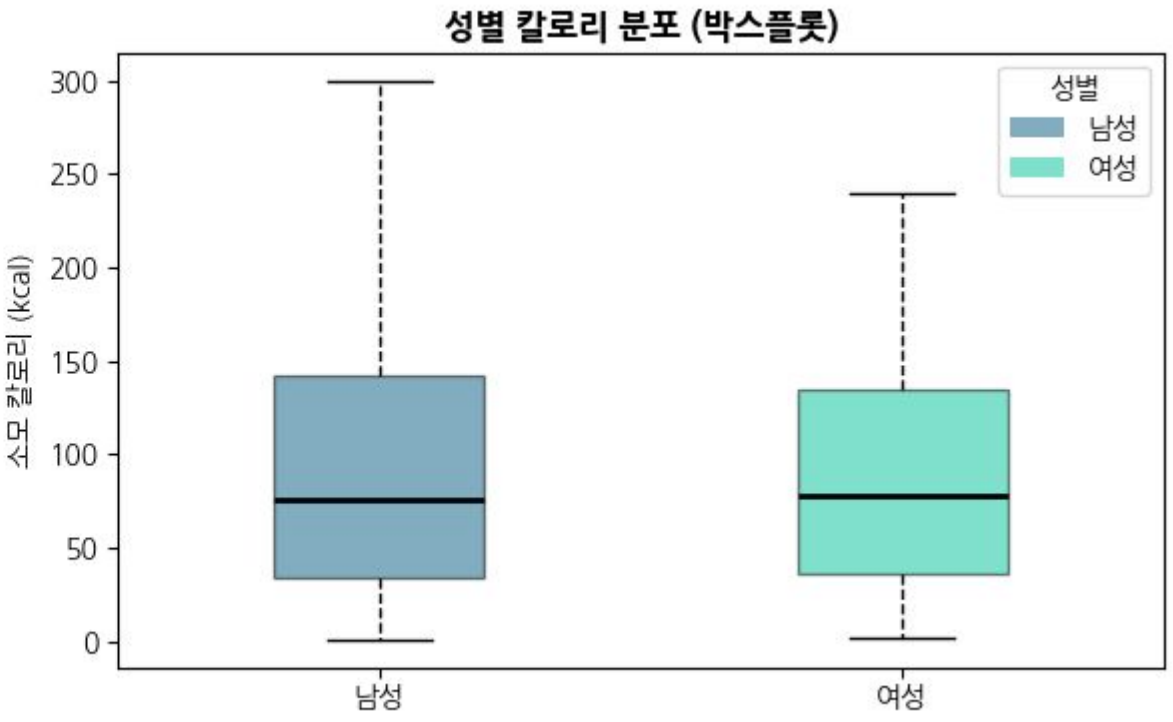

Weight_Status vs Calories_Burned (Mean Calories_Burned)
Mean Calories_Burned
0
20
40
60
80
Normal Weight
Obese
Overweight
Weight_Status
<
<


Gender vs Calories_Burned (Mean Calories_Burned)
Mean Calories_Burned
0
20
40
60
80
F
M
Gender
<

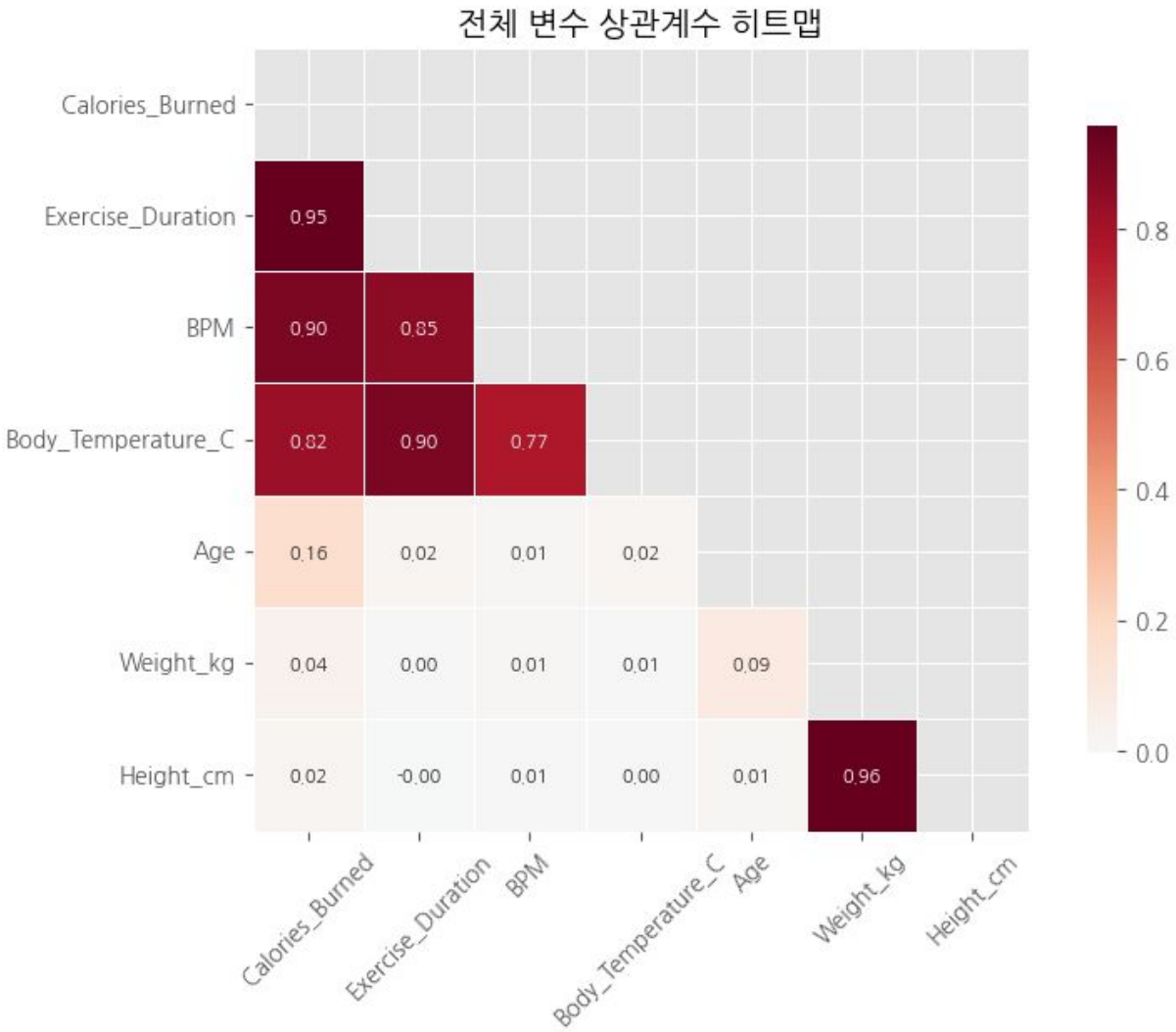

AgeGroup vs Calories_Burned (Mean Calories_Burned)
Mean Calories_Burned
0
20
40
60
80
100
0-20
21-30
31-40
41-50
51-60
60+
AgeGroup
<

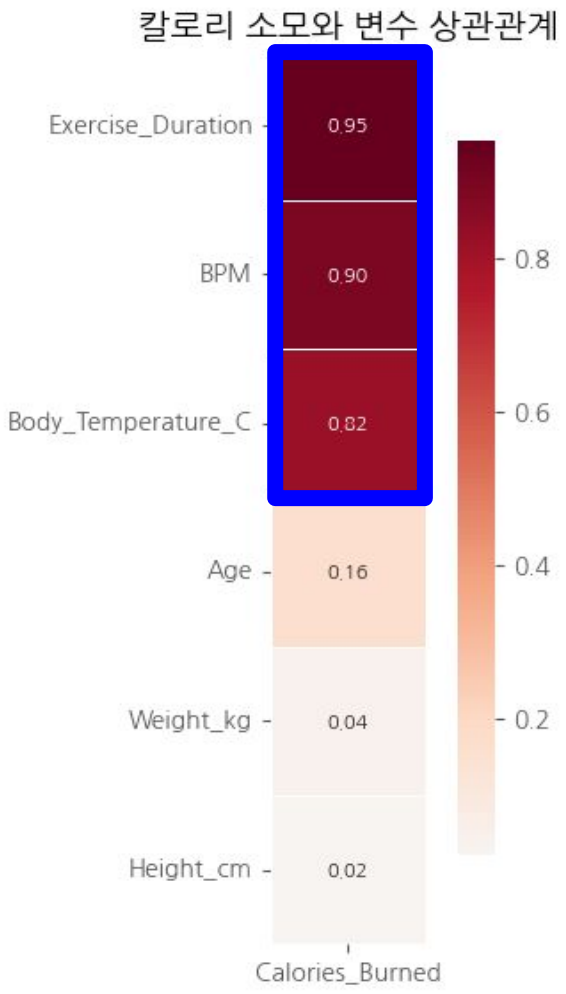
1.5 Box-plot (범주형)





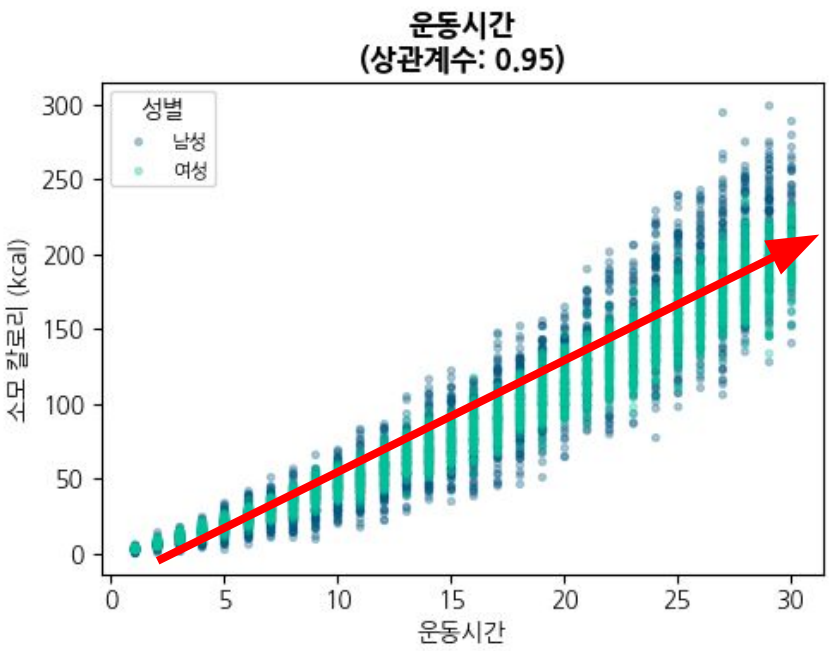
1.4 히트맵

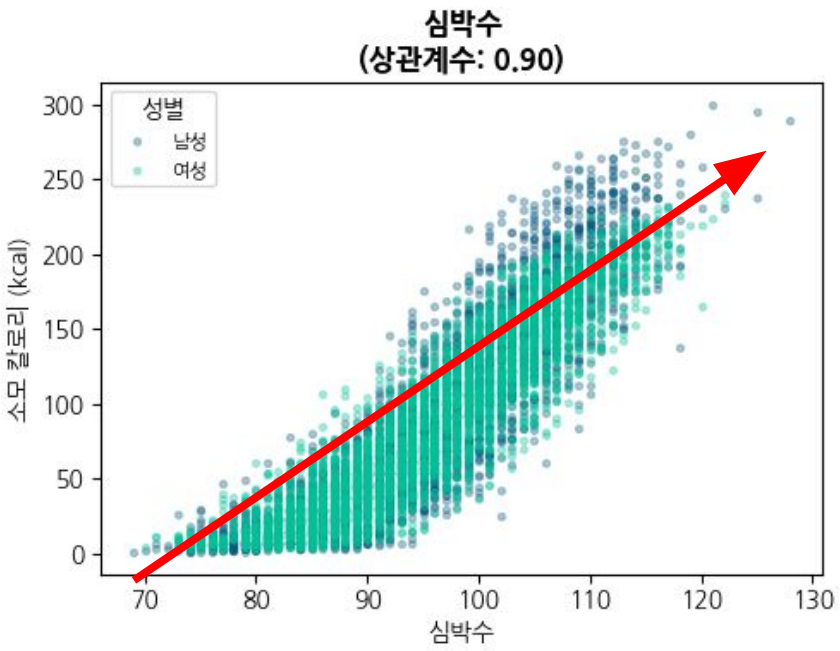

전체 변수 상관계수 히트맵
Calories_Burned
Exercise_Duration
BPM
Body_Temperature_C
Age
Weight_kg
Height_cm
0.95
0.90 0.85
0.82 0.90 0.77
0.16 0.02 0.01 0.02
0.04 0.00 0.01 0.01 0.09
0.02 -0.00 0.01 0.00 0.01 0.96
0.8
0.6
0.4
0.2
0.0

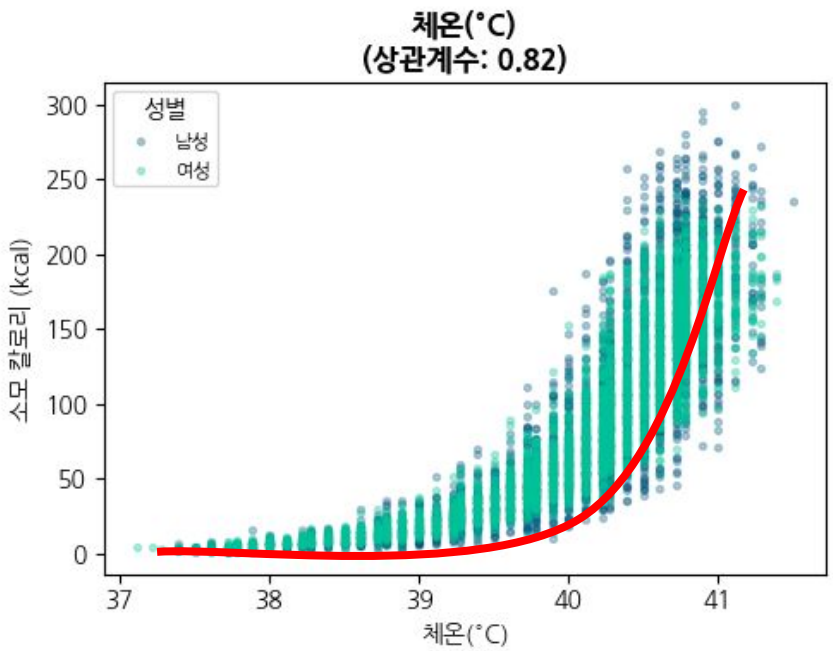

칼로리 소모와 변수 상관관계
Exercise_Duration
BPM
Body_Temperature_C
Age
Weight_kg
Height_cm
0.95
0.90
0.82
0.16
0.04
0.02
Calories_Burned
0.8
0.6
0.4
0.2

1.5 산점도(수치형)

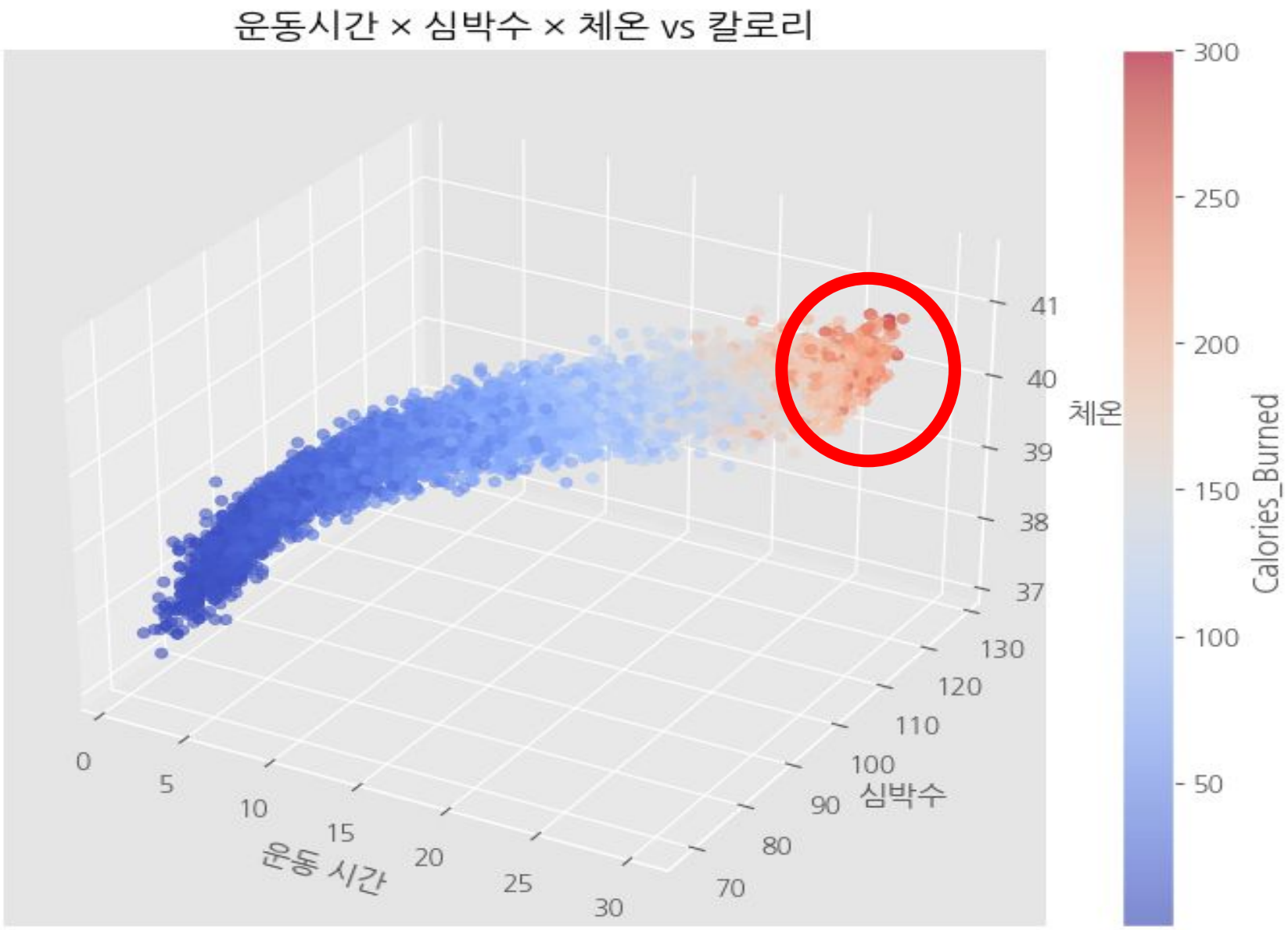
Top 3: 운동시간, 심박수, 체온







1.5 산점도 (다차항)



Top 3:

운동시간
심박수
체온

2. 데이터 전처리

2.1 단위 변환

```python
# Height, Weight, Body Temperature 단위 통일
for df in [train, test]:
    # 키 단위 변환: Feet + Inches → cm
    df['Height_cm']          = df['Height(Feet)']*30.48 + df['Height(Remainder_Inches)']*2.54
    # 몸무게 단위 변환: lb → kg
    df['Weight_kg']          = df['Weight(lb)'] * 0.453592
    # 체온 단위 변환: ℉ → ℃
    df['Body_Temperature_C'] = (df['Body_Temperature(F)'] - 32) * 5/9
    # 불필요한 원본 컬럼 제거
    df.drop(columns=['Height(Feet)','Height(Remainder_Inches)',
                     'Weight(lb)','Body_Temperature(F)'], inplace=True)

# Feature/Target 분리
y      = train['Calories_Burned'].values # 예측할 값 → 타겟
X      = train.drop(columns=['Calories_Burned','ID'])
X_test = test.drop(columns=['ID'])
```

2.2 피처 생성

```python
# ⚙️ 피처 생성 (물리 기반 + 다항식)
# ==============================================================
def build_base_features(df):
    D      = df['Exercise_Duration'].values
    BPM    = df['BPM'].values
    W      = df['Weight_kg'].values
    Age    = df['Age'].values
    T      = df['Body_Temperature_C'].values
    H      = df['Height_cm'].values
    Td     = T - 37.0
    mask_m = (df['Gender'] == 'M').values

    keytel = np.where(mask_m,
        (-55.0969 + 0.6309*BPM + 0.1988*W + 0.2017*Age) * D / 4.184,
        (-20.4022 + 0.4472*BPM - 0.1263*W + 0.074*Age)  * D / 4.184
    ).clip(0)

    bmr = np.where(mask_m,
        88.362  + 13.397*W + 4.799*H - 5.677*Age,
        447.593 +  9.247*W + 3.098*H - 4.330*Age
    )

    hr_ratio = BPM / np.maximum(220 - Age, 1)
    bsa      = np.sqrt(H * W / 3600)
```

2.2.1 심볼릭 네이밍

```python
# ⚙ 피처 생성 (물리 기반 + 다항식)
# ==========================================================
def build_base_features(df):
    D      = df['Exercise_Duration'].values
    BPM    = df['BPM'].values
    W      = df['Weight_kg'].values
    Age    = df['Age'].values
    T      = df['Body_Temperature_C'].values
    H      = df['Height_cm'].values
    Td     = T - 37.0
    mask_m = (df['Gender'] == 'M').values
```

가독성

+

물리적 직관

2.2.2 물리 및 생리학적 기반 피처

1. Keytel의 심박수 기반 칼로리 소모

고려시 높은 심박수 즉 강도 높은 운동일수록 칼로리 소모가 심할 것이다.

women. Factors found to have a significant interaction with energy expenditure included age, weight and gender. Mode of exercise (cycling versus

$$EE\,(\mathrm{kJ/min}) = Gender \times (-55.0969 + 0.6309 \times HR_{act} + 0.1988 \times Weight + 0.2017 \times Age + (1 - Gender) \times (-20.4022 + 0.4472 \times HR_{act} - 0.1263 \times Weight + 0.074 \times Age)$$

(Keytel et al, 2005)

```
keytel = np.where(mask_m,
    (-55.0969 + 0.6309*BPM + 0.1988*W + 0.2017*Age) * D / 4.184,
    (-20.4022 + 0.4472*BPM - 0.1263*W + 0.074*Age)  * D / 4.184
).clip(0)
```

2.2.2 물리 및 생리학적 기반 피처

2. Harris Benedict의 기초대사량(BMR) 공식

가설2. 운동 시, 운동으로 인한 칼로리 소모뿐만 아니라 기초대사량으로 인한 칼로리 소모도 발생한다.

Revised Formula (1984)

Often used for better accuracy, the 1984 revisions are: National Institutes of Health (.gov) +2

- **Men:**
  $$BMR = 88.362 + (13.397 \times \text{weight kg}) + (4.799 \times \text{height cm}) - (5.677 \times \text{age years})$$
- **Women:**
  $$BMR = 447.593 + (9.247 \times \text{weight kg}) + (3.098 \times \text{height cm}) - (4.330 \times \text{age years})$$

(Pavlidou et al, 2023)

```
bmr = np.where(mask_m,
        88.362  + 13.397*W + 4.799*H - 5.677*Age,
        447.593 +  9.247*W + 3.098*H - 4.330*Age
)
```

2.2.2 물리 및 생리학적 기반 피처

3. Fox & Haskell의 심혈관 공식

가설3. 나이가 들수록 운동으로 인한 칼로리 소모가 줄어들 것이다.

The Fox-Haskell formula is a widely used, simple equation to estimate maximum heart rate (HR_{max}):

$$\text{Max HR} = 220 - \text{Age}$$

(Nikolaidis et al, 2018)

```
hr_ratio = BPM / np.maximum(220 - Age, 1)
```

2.2.2 물리 및 생리학적 기반 피처

4. Mosteller의 체표면적(BSA) 공식

가설4. 신체 표면적이 클수록 칼로리 소모가 심할 것이다.

Fig. 15.1

$$\text{DuBois Formula: BSA } (m^2) = \sqrt{\frac{\text{Height (cm)} \times \text{Weight (kg)}}{3{,}600}}$$

Body surface area (BSA) according to Dubois formula [2]

(Schmidt et al, 2019)

```
bsa        = np.sqrt(H * W / 3600)
```

2.2.3 물리 및 생리학적 상호작용 피처

$$EE\,(\mathrm{kJ/min}) = Gender \times (-55.0969 + 0.6309 \times HR_{act} + 0.1988 \times Weight + 0.2017 \times Age + (1 - Gender) \times (-20.4022 + 0.4472 \times HR \; 0.1263 \times Weight + 0.074 \times Age)$$

Revised Formula (1984)

Often used for better accuracy, the 1984 revisions are: National Institutes of Health (.gov) +2

- **Men:**

$$3.362 + (13.397 \times \text{weight kg}) + (4.799 \times \text{height cm}) - (5.677 \times \text{age years})$$

$$47.593 + (9.247 \times \text{weight kg}) + (3.098 \times \text{height cm}) - (4.330 \times \text{age years})$$

The Fox-Haskell formula is a widely used, simple equation to estimate maximum heart rate (HR_{max}):

$$\text{Max HR} = 220 - \text{Age}$$

Fig. 15.1

$$\text{DuBois Formula: BSA } (m^2) = \sqrt{\frac{\text{Height (cm)} \times \text{Weight (kg)}}{3{,}600}}$$

Body surface area (BSA) according to Dubois formula [2]

계수 및 가중치 고정

과연 공식만으로 100% 설명할 수 있을까?

2.2.3 물리 및 생리학적 상호작용 피처

$$E_{(\text{에너지})} = p_{(\text{일률})} \times t_{(\text{시간})}$$

↓

$$E_{(\text{칼로리 소모 에너지})} = p_{(\text{에너지가 소비되는 속도})} \times t_{(\text{해당 강도가 지속된 시간})}$$

체중
심박수
체온
...

운동시간

2.2.3 물리 및 생리학적 상호작용 피처

운동시간 (0.95)

BPM = 그냥 심박수

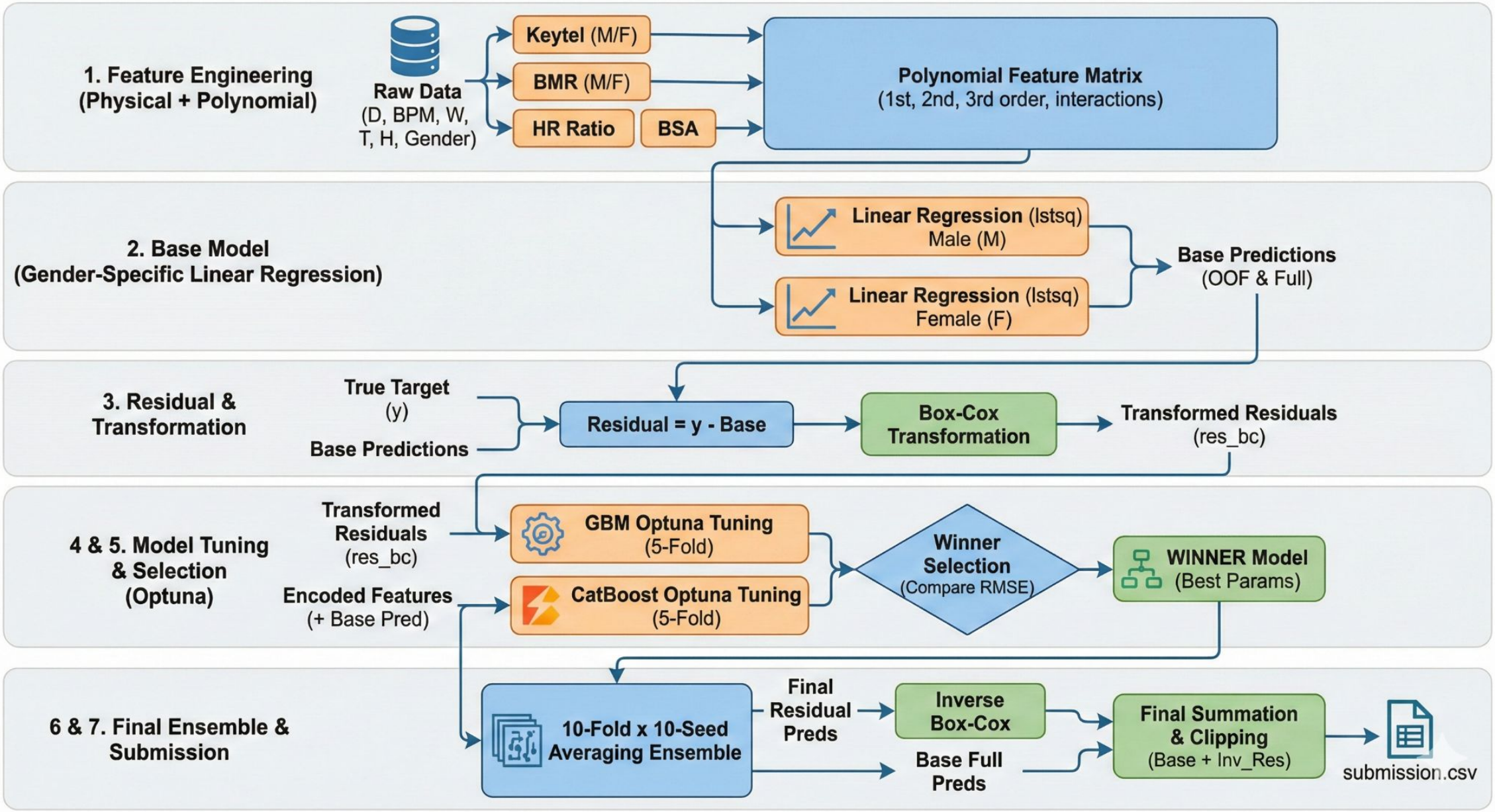
BPM x 운동시간 = 시간 비례 심박수 강도

모든 항에 운동시간(D) 곱함

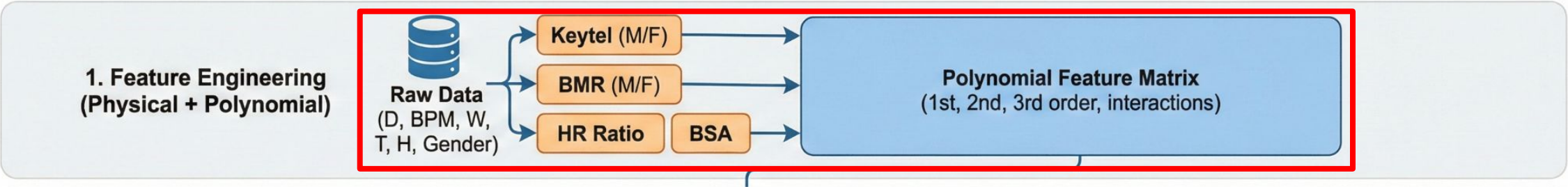
```
return np.column_stack([
    # 1차
    BPM*D, T*D, Td*D, W*D, Age*D, H*D, D, np.ones(len(df)),
    # 2차
    BPM*Td*D, BPM*W*D, Td*W*D, BPM*Age*D, Td*Age*D, H*W*D,
    # 제곱항
    BPM**2*D, Td**2*D, W**2*D,
    # 3차
    BPM*Td*W*D,
    # keytel 관련 다항식
    keytel, keytel*Td, keytel*hr_ratio,
    # bmr관련
    bmr*D, bmr*hr_ratio*D,
    # 심박수 관련
    hr_ratio*D, hr_ratio*W*D, hr_ratio*Td*D,
    # bsa(신체표면적)
    bsa*D, bsa*BPM*D,
])
```

3. 모델링

3.1 전체 모델 구조도


1. Feature Engineering
(Physical + Polynomial)
Raw Data
(D, BPM, W, T, H, Gender)
Keytel (M/F)
BMR (M/F)
HR Ratio
BSA
Polynomial Feature Matrix
(1st, 2nd, 3rd order, interactions)
2. Base Model
(Gender-Specific Linear Regression)
Linear Regression (lstsq)
Male (M)
Linear Regression (lstsq)
Female (F)
Base Predictions
(OOF & Full)
3. Residual &
Transformation
True Target
(y)
Base Predictions
Residual = y - Base
Box-Cox
Transformation
Transformed Residuals
(res_bc)
4 & 5. Model Tuning
& Selection
(Optuna)
Transformed
Residuals
(res_bc)
Encoded Features
(+ Base Pred)
GBM Optuna Tuning
(5-Fold)
CatBoost Optuna Tuning
(5-Fold)
Winner
Selection
(Compare RMSE)
WINNER Model
(Best Params)
6 & 7. Final Ensemble &
Submission
10-Fold x 10-Seed
Averaging Ensemble
Final
Residual
Preds
Inverse
Box-Cox
Base Full
Preds
Final Summation
& Clipping
(Base + Inv_Res)
submission.csv

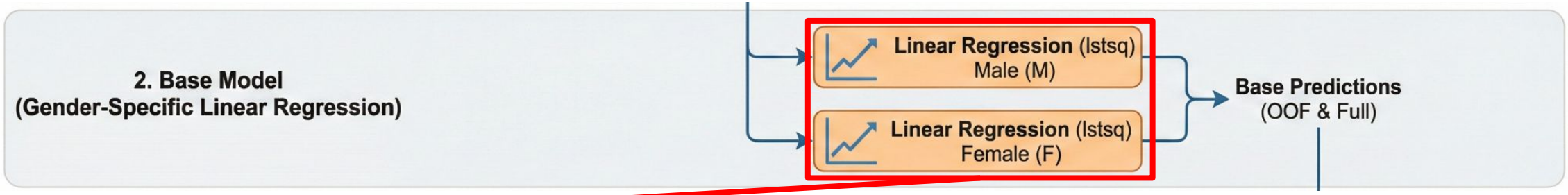
3.2 피처 엔지니어링




```python
# ⚙️ 피처 생성 (물리 기반 + 다항식)
# ============================================================
def build_base_features(df):
    D      = df['Exercise_Duration'].values
    BPM    = df['BPM'].values
    W      = df['Weight_kg'].values
    Age    = df['Age'].values
    T      = df['Body_Temperature_C'].values
    H      = df['Height_cm'].values
    Td     = T - 37.0
    mask_m = (df['Gender'] == 'M').values

    keytel = np.where(mask_m,
        (-55.0969 + 0.6309*BPM + 0.1988*W + 0.2017*Age) * D / 4.184,
        (-20.4022 + 0.4472*BPM - 0.1263*W + 0.074*Age)  * D / 4.184
    ).clip(0)

    bmr = np.where(mask_m,
        88.362  + 13.397*W + 4.799*H - 5.677*Age,
        447.593 +  9.247*W + 3.098*H - 4.330*Age
    )

    hr_ratio = BPM / np.maximum(220 - Age, 1)
    bsa      = np.sqrt(H * W / 3600)
```

```python
return np.column_stack([
    # 1차
    BPM*D, T*D, Td*D, W*D, Age*D, H*D, D, np.ones(len(df)),
    # 2차
    BPM*Td*D, BPM*W*D, Td*W*D, BPM*Age*D, Td*Age*D, H*W*D,
    # 제곱항
    BPM**2*D, Td**2*D, W**2*D,
    # 3차
    BPM*Td*W*D,
    # keytel 관련 다항식
    keytel, keytel*Td, keytel*hr_ratio,
    # bmr관련
    bmr*D, bmr*hr_ratio*D,
    # 심박수 관련
    hr_ratio*D, hr_ratio*W*D, hr_ratio*Td*D,
    # bsa(신체표면적)
    bsa*D, bsa*BPM*D,
])
```

3.3 Base 모델




```python
# ============================================================
# ⚙️ Base 모델 학습
# ============================================================
def base_oof(tr_df, y, kf):
    oof = np.zeros(len(tr_df))
    for tr_idx, val_idx in kf.split(tr_df):
        for g in ['F', 'M']:
            tr_m  = (tr_df.iloc[tr_idx]['Gender'] == g).values
            val_m = (tr_df.iloc[val_idx]['Gender'] == g).values
            if tr_m.sum() == 0 or val_m.sum() == 0: continue
            c,_,_,_ = lstsq(build_base_features(tr_df.iloc[tr_idx][tr_m]),
                            y[tr_idx][tr_m], rcond=None)
            oof[val_idx[val_m]] = build_base_features(tr_df.iloc[val_idx][val_m]) @ c
    return oof

def base_full(tr_df, te_df, y):
    pred = np.zeros(len(te_df))
    for g in ['F', 'M']:
        tr_m = (tr_df['Gender'] == g).values
        te_m = (te_df['Gender'] == g).values
        c,_,_,_ = lstsq(build_base_features(tr_df[tr_m]), y[tr_m], rcond=None)
        pred[te_m] = build_base_features(te_df[te_m]) @ c
    return pred

print("📐 Base 모델 학습 중...")
kf10       = KFold(n_splits=10, shuffle=True, random_state=42)
train_base = base_oof(X, y, kf10)
test_base  = base_full(X, X_test, y)
base_rmse  = np.sqrt(mean_squared_error(y, train_base))
print(f"   Base 모델 OOF RMSE : {base_rmse:.6f}")
```

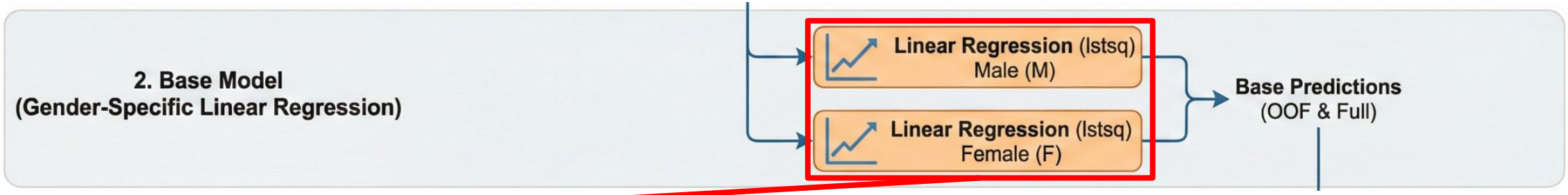
lstsq (최소제곱법)

타겟예측선형방정식

항 가중치 계수

$$\hat{y} = \underbrace{c_1(BPM \cdot D) + c_2(T \cdot D) + \cdots + c_7(D)}_{\text{1차 물리 상호작용}}$$
$$+ \underbrace{c_8(BPM \cdot T_d \cdot D) + \cdots + c_{13}(H \cdot W \cdot D)}_{\text{2차 비선형 상호작용}}$$
$$+ \underbrace{c_{14}(BPM^2 \cdot D) + \cdots + c_{16}(W^2 \cdot D)}_{\text{제곱항 (곡률 반영)}}$$
$$+ \underbrace{c_{17}(Keytel) + c_{18}(Keytel \cdot T_d) + \ldots}_{\text{도메인 지식(Keytel) 보정}}$$
$$+ \underbrace{c_{22}(BMR \cdot D) + c_{23}(BMR \cdot hr_ratio \cdot D)}_{\text{기초대사량 기반 보정}}$$
$$+ \text{Intercept}(c_0)$$

3.3 Base 모델




```python
# ============================================================
# ⚙ Base 모델 학습
# ============================================================
def base_oof(tr_df, y, kf):
    oof = np.zeros(len(tr_df))
    for tr_idx, val_idx in kf.split(tr_df):
        for g in ['F', 'M']:
            tr_m  = (tr_df.iloc[tr_idx]['Gender'] == g).values
            val_m = (tr_df.iloc[val_idx]['Gender'] == g).values
            if tr_m.sum() == 0 or val_m.sum() == 0: continue
            c,_,_,_ = lstsq(build_base_features(tr_df.iloc[tr_idx][tr_m]),
                            y[tr_idx][tr_m], rcond=None)
            oof[val_idx[val_m]] = build_base_features(tr_df.iloc[val_idx][val_m]) @ c
    return oof

def base_full(tr_df, te_df, y):
    pred = np.zeros(len(te_df))
    for g in ['F', 'M']:
        tr_m = (tr_df['Gender'] == g).values
        te_m = (te_df['Gender'] == g).values
        c,_,_,_ = lstsq(build_base_features(tr_df[tr_m]), y[tr_m], rcond=None)
        pred[te_m] = build_base_features(te_df[te_m]) @ c
    return pred

print("📐 Base 모델 학습 중...")
kf10       = KFold(n_splits=10, shuffle=True, random_state=42)
train_base = base_oof(X, y, kf10)
test_base  = base_full(X, X_test, y)
base_rmse  = np.sqrt(mean_squared_error(y, train_base))
print(f"   Base 모델 OOF RMSE : {base_rmse:.6f}")
```

Gender-Specific

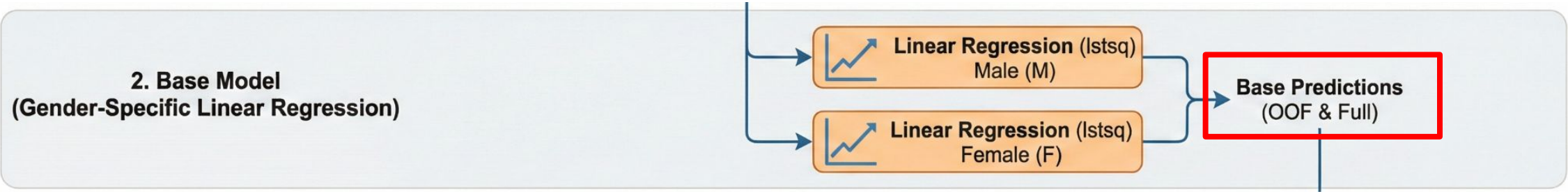
1. 균등한 데이터 분포
여성 : 3769명 (50.3%)
남성 : 3731명 (49.7%)
거의 1:1

2. 범주형 피처

3. 기존 물리기반 피처성별에 따른 생리학적 특성 고려

4. 성별(2) X 체중 상태(3)..?

3.3 Base 모델

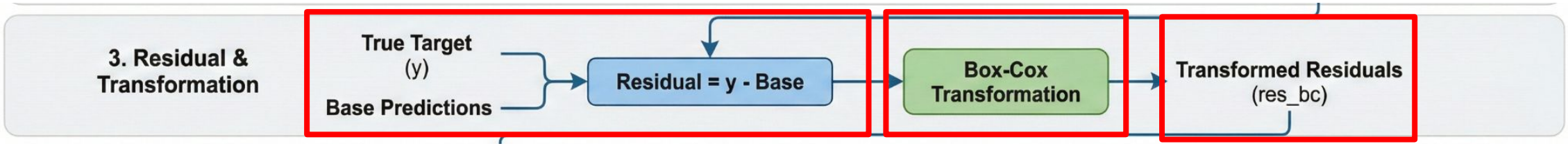



```
print("📐 Base 모델 학습 중...")
kf10       = KFold(n_splits=10, shuffle=True, random_state=42)
train_base = base_oof(X, y, kf10)
test_base  = base_full(X, X_test, y)
base_rmse  = np.sqrt(mean_squared_error(y, train_base))
print(f"   Base 모델 OOF RMSE : {base_rmse:.6f}")
```

10-Fold …… train OOF / test OOF !

```
📐 다항식 기저 모델 학습 중...
   기저 모델 OOF RMSE: 0.290558
```

3.4 잔차 + Box-Cox

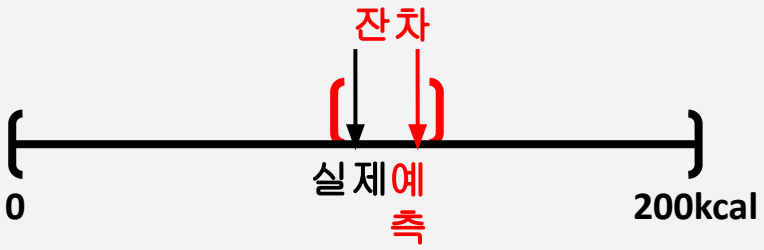



```python
# ============================================================
# STEP 2: 잔차 + Box-Cox
# ============================================================
residual = y - train_base
if (residual <= 0).any():
    res_shift = float(1 - residual.min())
    res_bc, lambda_ = stats.boxcox(residual + res_shift)
else:
    res_shift = 0.0
    res_bc, lambda_ = stats.boxcox(residual)

def inv_bc(v, l):
    return np.exp(v) if l == 0 else np.power(np.maximum(v*l + 1, 1e-10), 1/l)

def raw_rmse(val_idx, oof_res):
    pred = np.clip(train_base[val_idx] + inv_bc(oof_res, lambda_) - res_shift, 1.0, None)
    return np.sqrt(mean_squared_error(y[val_idx], pred))
```

잔차 (Residual)

1. 실제값 - 예측값

2. 미세한 오차 집중 공략

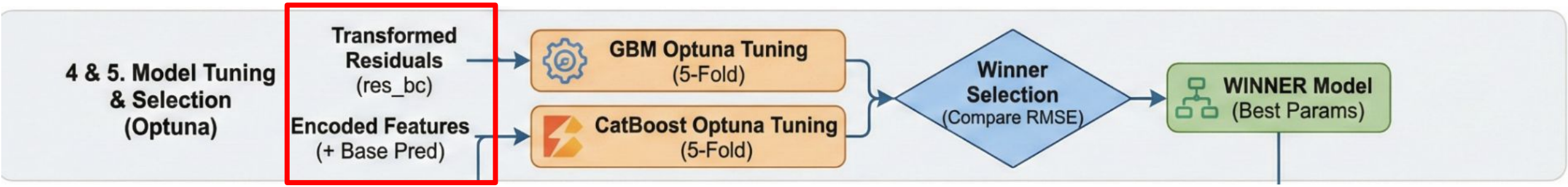


-> 예측값(거시) + 잔차(미시)

3. Box-Cox (잔차의 정규 분포 변환)

4. raw_rmse (역변환)

3.5 인코딩




```python
# ============================================================
# STEP 3: 피처 인코딩
# ============================================================
def encode_gbm(df, base_pred):
    d = df.copy()
    d['Gender'] = (d['Gender'] == 'M').astype(int)
    ws_map = {'Underweight':0,'Normal Weight':1,'Overweight':2,'Obese':3}
    d['Weight_Status'] = d['Weight_Status'].map(ws_map).fillna(1)
    d['Base'] = base_pred
    return d

def encode_cat(df, base_pred):
    d = df.copy()
    d['Base'] = base_pred
    return d

X_gbm    = encode_gbm(X,      train_base)
X_te_gbm = encode_gbm(X_test, test_base)
X_cat    = encode_cat(X,      train_base)
X_te_cat = encode_cat(X_test, test_base)

CAT_FEATURES = ['Gender', 'Weight_Status']

assert not X_gbm.isnull().any().any(), "❌ X_gbm NaN"
assert not X_cat.isnull().any().any(), "❌ X_cat NaN"
print("✅ 인코딩 검증 통과")
```

피처 인코딩

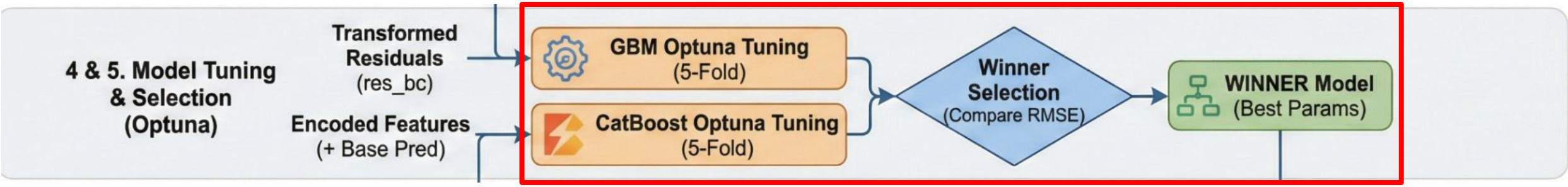
1. 모델별 맞춤 피처 인코딩 (GBM, CatBoost)

2. GBM -> 잔차 학습 최적화 모델

3. CatBoost -> 앙상블 조합 중 가장 좋은 성능 (범주형 피처 분류 뛰어남)

4. Gender, Weight_Status -> 범주형 피처 인코딩

3.6 모델 최적화




```python
# ============================================================
# STEP 4-A: GBM Optuna (n_trials=100)
# ============================================================
kf_opt = KFold(n_splits=5, shuffle=True, random_state=42)

def gb_obj(trial):
    p = dict(
        n_estimators     = trial.suggest_int('n_estimators', 300, 5000),
        learning_rate    = trial.suggest_float('learning_rate', 0.001, 0.1, log=True),
        max_depth        = trial.suggest_int('max_depth', 1, 4),
        subsample        = trial.suggest_float('subsample', 0.5, 1.0),
        min_samples_leaf = trial.suggest_int('min_samples_leaf', 1, 80),
        max_features     = trial.suggest_float('max_features', 0.3, 1.0),
        random_state     = 42,
    )
    scores = []
    for tr, val in kf_opt.split(X_gbm):
        m = GradientBoostingRegressor(**p)
        m.fit(X_gbm.iloc[tr], res_bc[tr])
        scores.append(raw_rmse(val, m.predict(X_gbm.iloc[val])))
    return np.mean(scores)

print("\n🔍 [1/2] GBM Optuna (n_trials=100)...")
gb_study = optuna.create_study(direction='minimize')
gb_study.optimize(gb_obj, n_trials=100, show_progress_bar=True)
gb_best  = gb_study.best_params
print(f"✅ GBM 최적 RMSE : {gb_study.best_value:.6f}")
```

Optuna

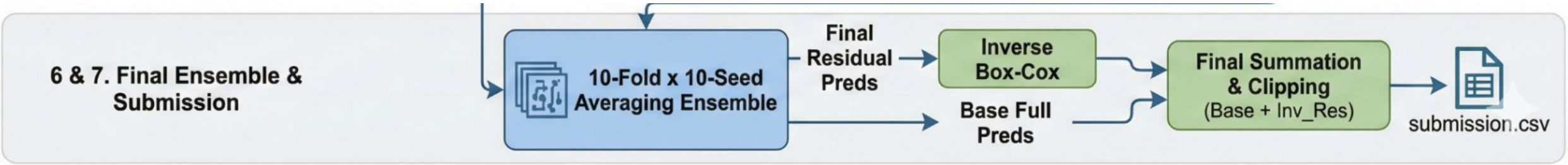
1. GBM 하이퍼 파라미터 최적화

```
📊 [GBM Best Hyperparameters]
------------------------------------
      n_estimators : 4732
     learning_rate : 0.0696205016732983
         max_depth : 3
         subsample : 0.7588787474157884
  min_samples_leaf : 7
      max_features : 0.9528867898010692
------------------------------------
         Best RMSE : 0.225704
```

2. VS CatBoost

```
==============================================
  GBM      Optuna RMSE : 0.225704
  CatBoost Optuna RMSE : 0.290918
  🏆 승자: GBM (차이: +0.065214)
==============================================
```

3.7 최종 모델 학습 및 평가




```python
# ============================================================
# STEP 5: 최종 학습 (승자 모델 × 10-Fold × 10 Seeds)
# ============================================================
SEEDS = [42, 777, 2024, 1234, 99, 314, 5678, 9999, 111, 2077]
N     = 10
kf    = KFold(n_splits=N, shuffle=True, random_state=42)
oof_final  = np.zeros(len(y))
pred_final = np.zeros(len(test))
shap_model = None  # SHAP용 모델 저장 변수

print(f"\n=== 최종 학습: {WINNER} × 10-Fold × {len(SEEDS)} Seeds ===")

for seed in SEEDS:
    fold_oof = np.zeros(len(y))
    for fold, (tr_idx, val_idx) in enumerate(kf.split(X_gbm)):

        if WINNER == 'GBM':
            m = GradientBoostingRegressor(**{**gb_best, 'random_state': seed})
            m.fit(X_gbm.iloc[tr_idx], res_bc[tr_idx])
            fold_oof[val_idx]  = m.predict(X_gbm.iloc[val_idx])
            pred_final        += m.predict(X_te_gbm) / (len(SEEDS) * N)

            # SHAP용: 첫 번째 seed, 첫 번째 fold 모델만 저장
            if seed == 42 and fold == 0:
                shap_model = m
```

최종 모델 평가

1. 10-Fold X 10-Seed = 총 100번의 모델 생성 및 학습

2. GBM 진 최종 예측

```
=== 최종 학습: GBM × 10-Fold × 10 Seeds ===
  Fold 01 | 0.21647
  Fold 02 | 0.21247
  Fold 03 | 0.20822
  Fold 04 | 0.21457
  Fold 05 | 0.22039
  Fold 06 | 0.21333
  Fold 07 | 0.22047
  Fold 08 | 0.20994
  Fold 09 | 0.21561
  Fold 10 | 0.21651
  Seed 42 완료
  Seed 777 완료
  Seed 2024 완료
  Seed 1234 완료
  Seed 99 완료
  Seed 314 완료
  Seed 5678 완료
  Seed 9999 완료
  Seed 111 완료
  Seed 2077 완료

============================================
기저 모델 RMSE  : 0.290979
최종 OOF RMSE   : 0.213363
```

4. 모델 해석 및 시각화

4.1 SHAP

```python
# SHAP용: 첫 번째 seed, 첫 번째 fold 모델만 저장
if seed == 42 and fold == 0:
    shap_model = m
```

첫번째 모델(대표)
저장
시간단축용

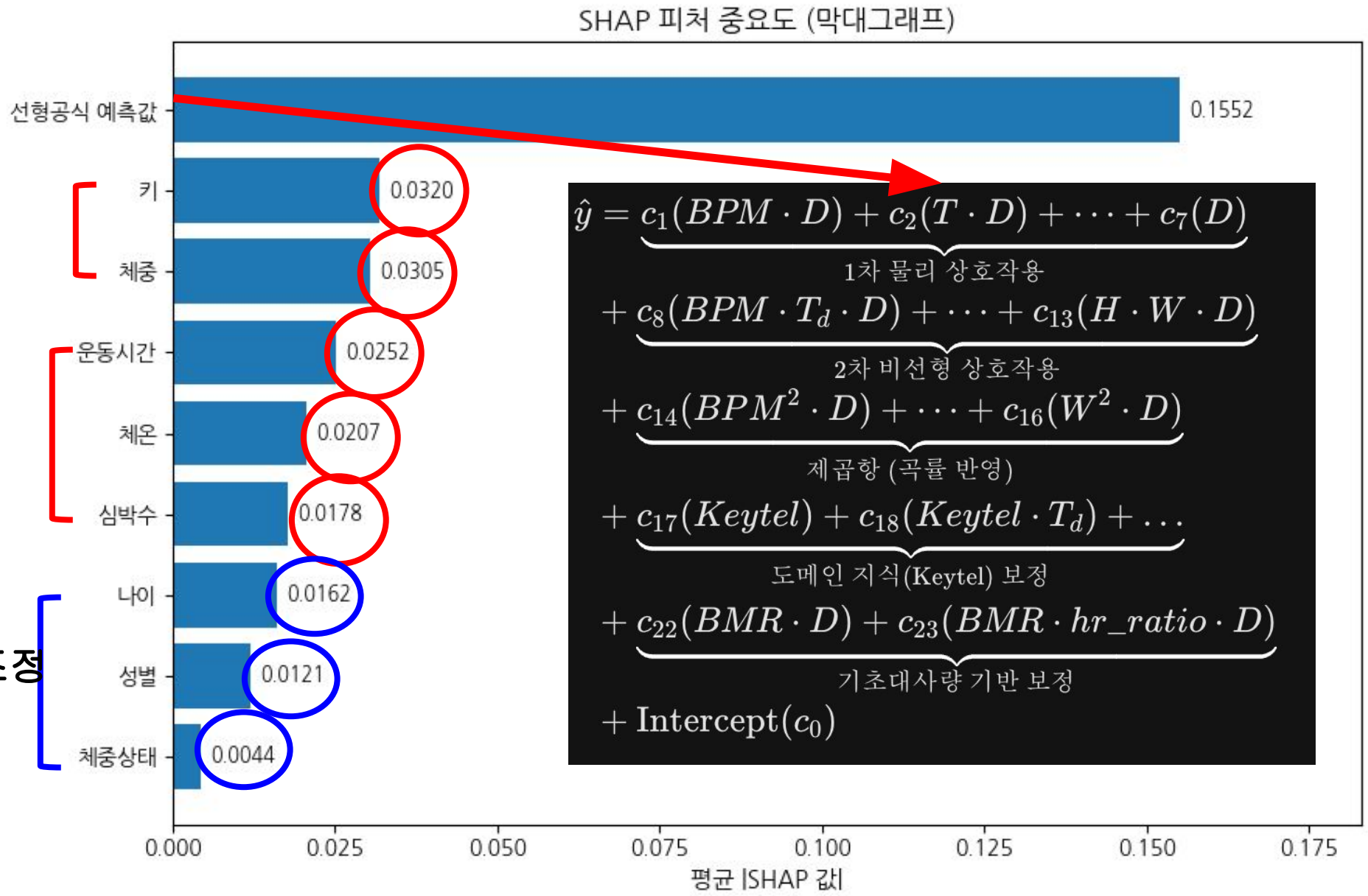
```python
# ============================================================
# 📊 SHAP 시각화
# ============================================================
print("\n📊 SHAP 계산 중...")

X_shap = X_gbm if WINNER == 'GBM' else X_cat

explainer   = shap.TreeExplainer(shap_model)
shap_values = explainer.shap_values(X_shap)

# 그래프 1: 피처 중요도 막대그래프
print("\n[1/2] 피처 중요도 (막대그래프)")
shap.summary_plot(shap_values, X_shap, plot_type='bar', show=True)

# 그래프 2: Beeswarm (방향 + 크기 동시에)
print("\n[2/2] Beeswarm (각 변수의 방향과 영향력)")
shap.summary_plot(shap_values, X_shap, show=True)

print("\n✅ SHAP 시각화 완료!")
print("   ※ SHAP은 seed=42, fold=1 모델 기준 (예측 성능과 무관, 피처 해석용)")
```

4.1 SHAP(피처 중요도)

개인화
보정

추가
칼로리
소모 조정

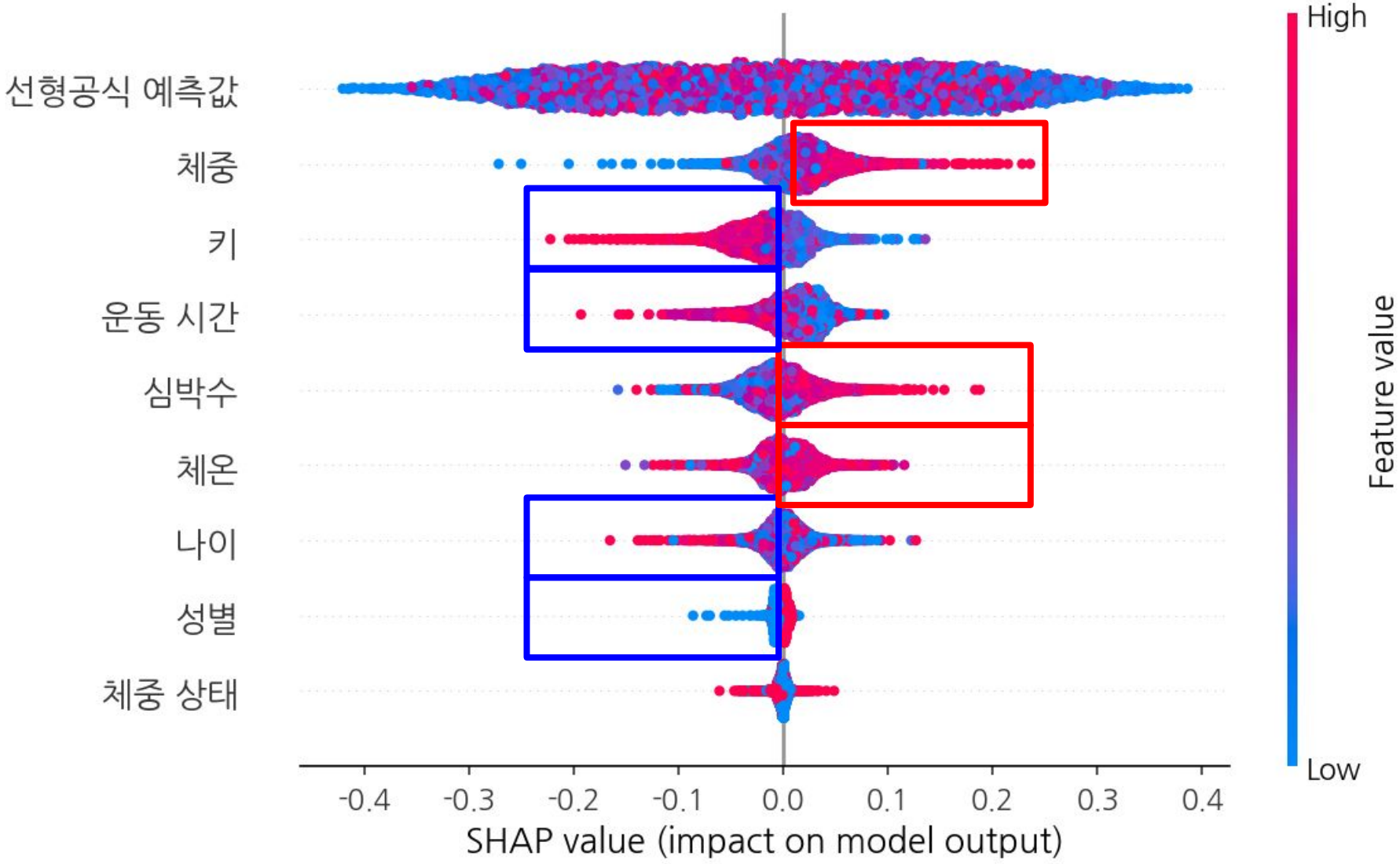
잔차 미세 조정
기여 낮음



$$\hat{y} = \underbrace{c_1(BPM \cdot D) + c_2(T \cdot D) + \cdots + c_7(D)}_{\text{1차 물리 상호작용}}$$
$$+ \underbrace{c_8(BPM \cdot T_d \cdot D) + \cdots + c_{13}(H \cdot W \cdot D)}_{\text{2차 비선형 상호작용}}$$
$$+ \underbrace{c_{14}(BPM^2 \cdot D) + \cdots + c_{16}(W^2 \cdot D)}_{\text{제곱항 (곡률 반영)}}$$
$$+ \underbrace{c_{17}(Keytel) + c_{18}(Keytel \cdot T_d) + \ldots}_{\text{도메인 지식(Keytel) 보정}}$$
$$+ \underbrace{c_{22}(BMR \cdot D) + c_{23}(BMR \cdot hr_ratio \cdot D)}_{\text{기초대사량 기반 보정}}$$
$$+ \text{Intercept}(c_0)$$

4.1 SHAP(Summary Plot)

lstsq(Base)이 틀린 부분을 GBM(Meta)이 어떻게 조절하는가 ?



균등
-> 모든 데이터 흐름 조정
->선형공식 예측값

우측 + 빨강색
-> 칼로리 더 높게 조정
-> 체중
-> 심박수
-> 체온

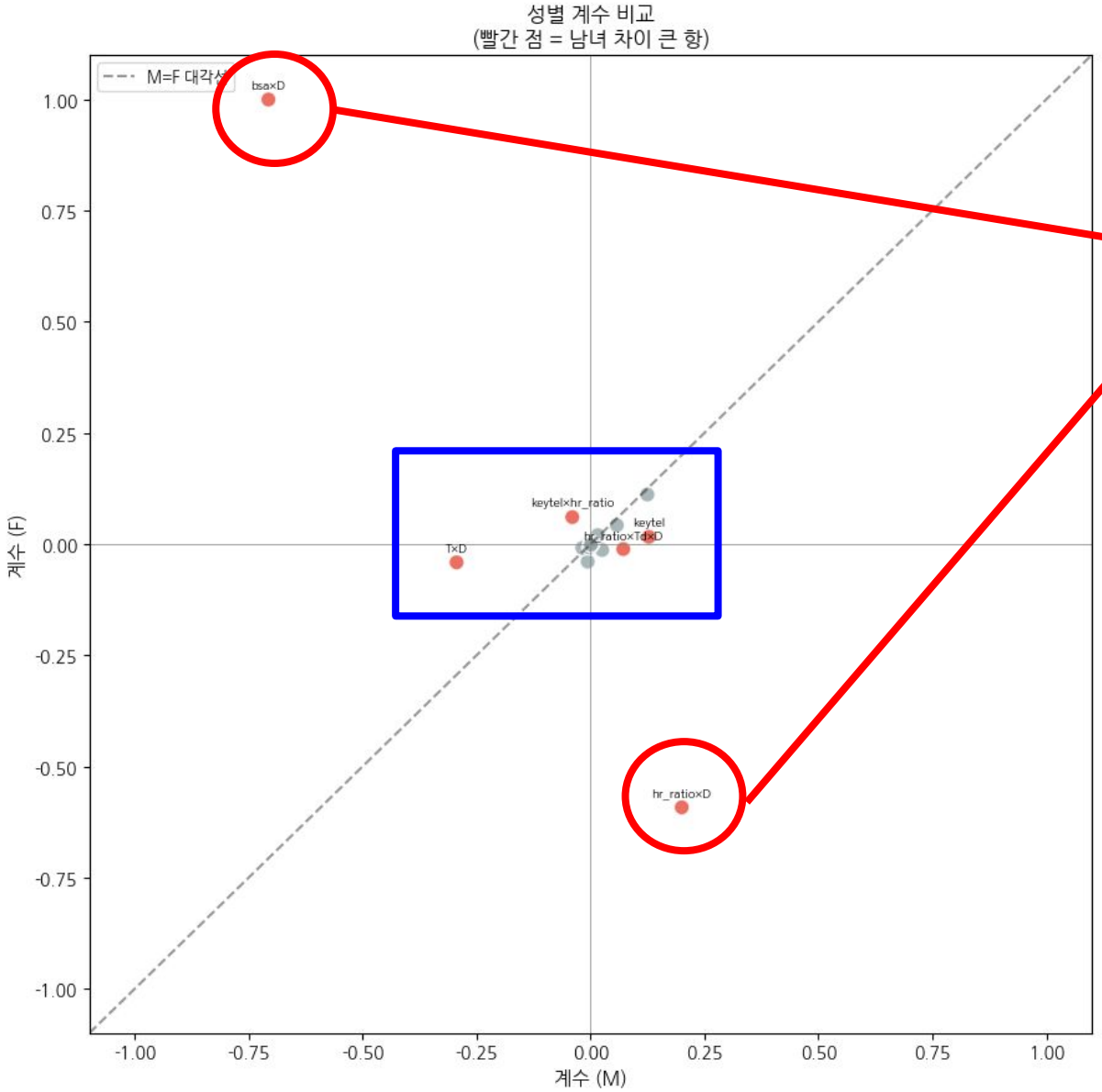
좌측 + 빨강색
-> 칼로리 더 낮게 조정
-> 키
-> 운동시간
-> 나이

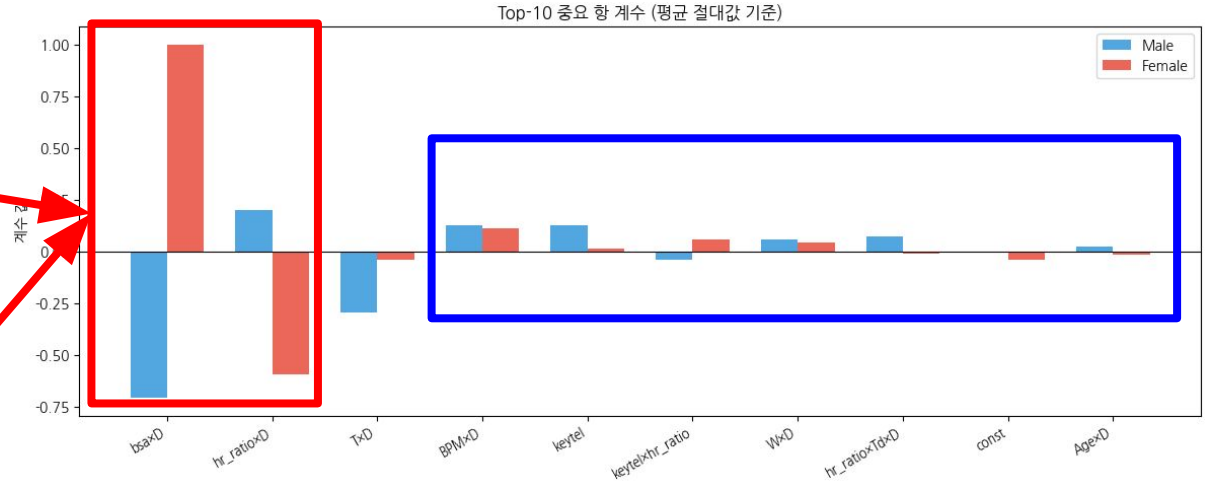
애매
-> 체중상태
-> 성별

5. 고찰

5.1 고찰

1. 다중공선성의 불완전한 해결





1,3 사분면 -> 같은 방향 (다중공선성 없음. 정상)

2,4 사분면 -> 부호 역전 (다중공선성 있음. 문제)

대각선 위 -> 남녀 계수 동일

키와 체중의 상관관계(**0.96**) -> 둘 중 뭘써도 오차 줄어드는 문제 발생 -> 키,체중 삭제하고 **BMI**를 추가했다면 ...

5.1 고찰

2. 중복피처의 과도한 생성

```
return np.column_stack([
    # 1차
    BPM*D, T*D, Td*D, W*D, Age*D, H*D, D, np.ones(len(df)),
    # 2차
    BPM*Td*D, BPM*W*D, Td*W*D, BPM*Age*D, Td*Age*D, H*W*D,
    # 제곱항
    BPM**2*D, Td**2*D, W**2*D,
    # 3차
    BPM*Td*W*D,
    # keytel 관련 다항식
    keytel, keytel*Td, keytel*hr_ratio,
    # bmr관련
    bmr*D, bmr*hr_ratio*D,
    # 심박수 관련
    hr_ratio*D, hr_ratio*W*D, hr_ratio*Td*D,
    # bsa(신체표면적)
    bsa*D, bsa*BPM*D,
])
```

W*D를 포함한 피처만 5개 이상

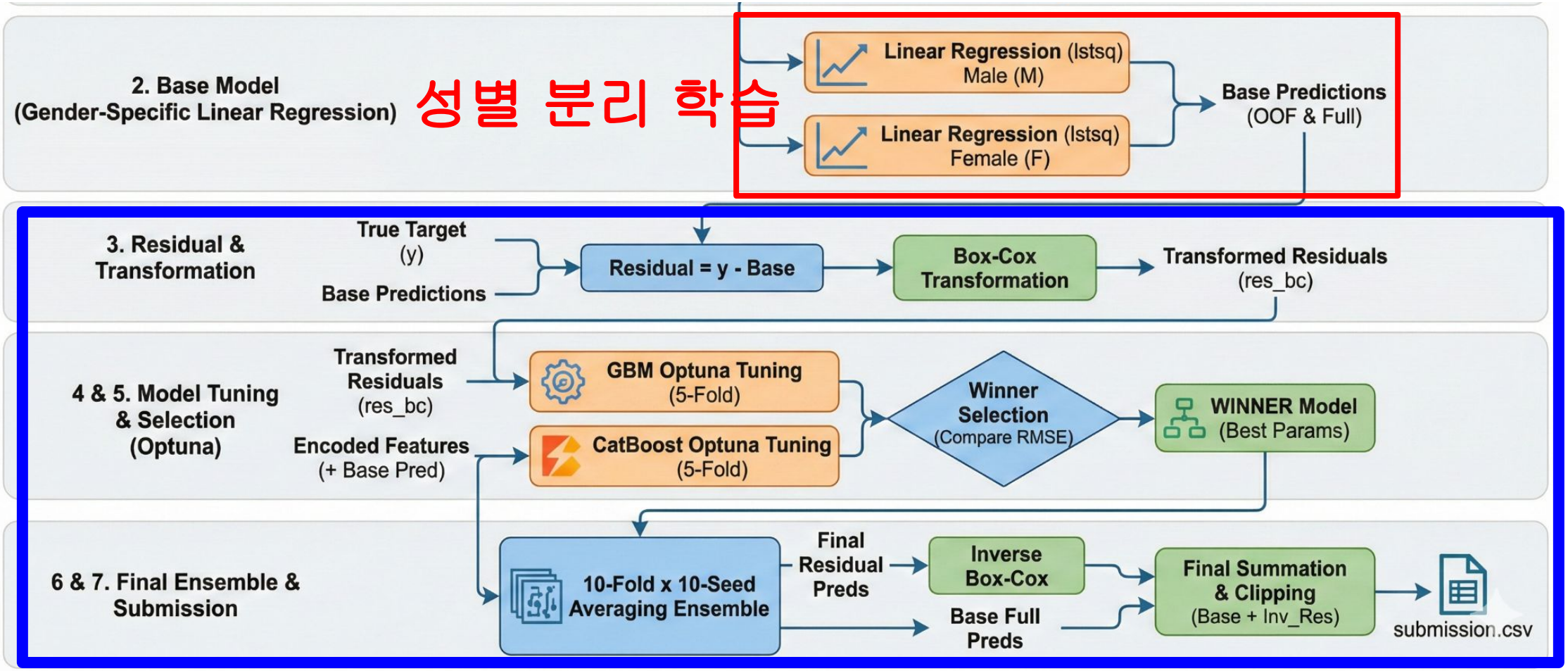
-> 피처간 독립성 전제하는 선형모델(lstsq)을 불안하게 만듦

-> 다중 공선성 발생 가능

-> 중복을 최대한 피했더라면 ...

5.1 고찰

3. 성별 분리 학습의 애매한 스탠스



성별 통합 학습 진행했다면 ...

출처

- Keytel, L. R., Goedecke, J. H., Noakes, T. D., Hiiloskorpi, H., Laukkanen, R., van der Merwe, L., & Lambert, E. V. (2005). Prediction of energy expenditure from heart rate monitoring during submaximal exercise. *Journal of sports sciences*, *23*(3), 289-297.
- Juul, A. J. A. E., & Jeukendrup, A. E. (2003). Heart rate monitoring: Applications and limitations. *Sports medicine*, *33*(7), 517-538.
- Galloway, S. D., & Maughan, R. J. (1997). Effects of ambient temperature on the capacity to perform prolonged cycle exercise in man. *Medicine & Science in Sports & Exercise*, *29*(9), 1240-1249.
- Schmidt, C. W. P. (2019). Body Surface Area (BSA). In *Pediatric Oncologic Pharmacy: A Complete Guide to Practice* (pp. 141-141). Cham: Springer International Publishing.
- Pavlidou E, Papadopoulou SK, Seroglou K, Giaginis C. Revised Harris-Benedict Equation: New Human Resting Metabolic Rate Equation. Metabolites. 2023 Jan 28;13(2):189. doi: 10.3390/metabo13020189. PMID: 36837808; PMCID: PMC9967803.
- Nikolaidis PT, Rosemann T, Knechtle B. Age-Predicted Maximal Heart Rate in Recreational Marathon Runners: A Cross-Sectional Study on Fox's and Tanaka's Equations. Front Physiol. 2018 Mar 15;9:226. doi: 10.3389/fphys.2018.00226. PMID: 29599724; PMCID: PMC5862813.

소감